

Class 4 — Know Your Weak Spots: Network Scanning & Vulnerability Analysis

CS 474 — Computer Security (3 cr) · 2026-2027 · Hour 1 lecture · Thursday, October 15, 2026

Easttom, Ch. 11 · LO 2 and LO 1 · Binder Part 4 — Vulnerability Assessment Report (due Wed Oct 21, 23:59)

Educational material — Cyber.SoHo / Asen Grozdanov. © 2026 Cyber.SoHo. All rights reserved. For use in an isolated, authorized lab environment only. Full terms: Cyber.SoHo Legal Notice, Terms of Use and Acceptable Use Policy v2.0 — https://cyber-soho.github.io/prevmon/00.01.%20cyber.soho%20-%20legal.html

STUDENT EDITION · From the Industry to the Classroom — Building the IT (Information Technology)'s Future, Today and Together !

Today in 60 minutes — five lessons

Lesson	Clock	Sub-topic	Micro-activity
1	0:00–0:11	Authorization, scope & rules of engagement	Fun quiz (3 min)
2	0:11–0:27	Port & service scanning with Nmap	Visualization (4 min)
3	0:27–0:43	Vulnerability scanning, CVE (Common Vulnerabilities and Exposures) & CVSS (Common Vulnerability Scoring System) (computational)	Short computational (4 min)
4	0:43–0:51	SAST (Static Application Security Testing), DAST (Dynamic Application Security Testing) & penetration testing	Flash-card blitz (2 min)
5	0:51–1:00	From findings to fixes	Fun quiz (2 min)

Before we start — the legal notice and the scope of this course

Authorization, not intent

Every scan and test in this course runs only on the lab systems and the fictional BlueFern scenario, with explicit written authorization. Unauthorized access is an offence — Criminal Code s. 342.1, **US** (United States) **CFAA** (Computer Fraud and Abuse Act), **EU** (European Union) Directive 2013/40/EU, Spain Ley Orgánica 10/1995 — regardless of intent.

No real targets, ever

No scanning of anything you do not own or hold written permission to test. No **OSINT** (Open-Source Intelligence) on real people or real organizations.

Computer security

CS 474 studies the machines, the software on them and the data they hold. Scanning is taught so that weaknesses can be found and fixed before an attacker finds them.

Full terms

Cyber.SoHo Legal Notice, Terms of Use and Acceptable Use Policy v2.0 — sections 6, 7.6, 7.7, 8 and 9; Annex C before clicking any link; Annex D for any real network.

Cyber.SoHo Legal Notice (v2.0)

The one FACT — and Binder Part 4

Your weekly work IS the project

and the project IS your exam preparation. Mid-Term (Nov 19) and Final (Jan 21) are milestones of the same BlueFern investigation.

Binder Part 4 — Vulnerability Assessment Report

Scope & authorization statement · scan summary · top-5 table (**CVE** (Common Vulnerabilities and Exposures) / **CVSS** (Common Vulnerability Scoring System) / priority / fix) · tool-comparison table. Due Wednesday, October 21, 23:59.

Hours 2–3 today (summary)

Lab 4.1 scan and record · Lab 4.2 top-5 findings with **CVE**, **CVSS**, priority and fix · Lab 4.3 **SAST** (Static Application Security Testing) vs **DAST** (Dynamic Application Security Testing) vs penetration test. The lab sheets are separate documents.

At home — 2 hours

Easttom, Chapter 11; finish and submit Part 4.

Lesson 1 — Authorization, scope & rules of engagement

0:00–0:11 · what may be scanned, by whom, when — ethics and law first

Two definitions

- **Vulnerability assessment** — systematic identification, verification and ranking of weaknesses, without exploiting them: "what could be attacked, and how badly?"
- **Penetration test** — exploits selected weaknesses, under contract, to prove what an attacker could achieve
- Class 4 and the whole of Lab 4 are an **assessment**: nothing is exploited; the finding goes into the report, not into a tool
- The sentence that governs the hour: **the law looks at authorization, not at intent**

The law looks at authorization, not at intent

- **Canada — Criminal Code**: s. 342.1 unauthorized use of a computer · s. 342.2 possession of a device (including software) to commit it · s. 430(1.1) mischief in relation to computer data
- **United States** — Computer Fraud and Abuse Act, 18 U.S.C. § 1030
- **European Union** — Directive 2013/40/EU on attacks against information systems
- **Spain** — Ley Orgánica 10/1995 (Código Penal), arts. 197 and 264
- None of them asks whether you meant well — what separates an assessment from an offence is **written authorization** from the person entitled to give it

Criminal Code s. 342.1 · 18 U.S.C. § 1030 · Directive 2013/40/EU · Ley Orgánica 10/1995

The scope document (rules of engagement) — five things it must contain

- 1 · The systems **in scope**, named by address or host name
- 2 · The systems **expressly excluded**
- 3 · The **techniques permitted** and the techniques **prohibited**
- 4 · The **dates and hours** of the test window
- 5 · **Who to contact** if something breaks
- Rule: *if it is not named there, it is out of scope — a target that is reachable is not a target that is authorized* (Cyber.SoHo Legal Notice §7.7.2)
- Lab work: the laboratory guide is the document · a real network: Annex D, signed by someone entitled to bind the owner

A scan is an intrusive act, not an observation (§7.6)

- A port scan **writes to logs**, **triggers alerts** in the intrusion-detection system, **consumes resources** and **can crash fragile services** (printers, point-of-sale terminals, old controllers)
- From the receiving end it is **indistinguishable from the first stage of an attack** — and is routinely reported as one
- Scan only the **designated targets**, in the designated range, during the exercise — no "quick look" at a campus, residence or workplace network
- Never scan a range because it "looked empty", because it answered, or because it sits next to a designated target
- A **credentialed scan** needs the written authorization of the credentials' owner too · never publish raw output that identifies a real host

What "no" means — and our scope document for today

The owner says no, or does not answer

No authorization exists. Nothing happens.

The window closes at 15:55

The authorization has expired. Ask for a new window — never "finish quickly".

You land outside the scope by mistake

Stop immediately, note the time, report (§6.8, §7.7.5). Continuing "to confirm the finding" converts an accident into an offence.

Today's scope (Class 4 lab guide)

Range 10.10.40.0/24 · hosts 10.10.40.25 (lab web shop) and 10.10.40.30 (lab file server) · 13:15–15:55 · permitted: port scan, version detection, unauthenticated vulnerability scan, passive baseline scan; **OS** (Operating System) detection only if your lab option's guide says so · prohibited: exploitation, denial of service, credentialed scans without the sheet, active attack modes, social engineering

Schematic 1 — The scope document and the five gates

Schematic 1 — The scope document and the five gates (Lesson 1)

A scan is lawful only inside a written scope, a time window and a list of permitted techniques — the law looks at authorization, not at intent.

The scope document (rules of engagement)

Cyber.SoHo Legal Notice §7.7.2 — what it must contain

1. Systems in scope — named by address or host name
2. Systems expressly excluded
3. Techniques permitted · techniques prohibited
4. Dates and hours of the test window
5. Who to contact if something breaks

If it is not named there, it is out of scope. A target that is reachable is not a target that is authorized.

Every proposed action passes five gates, in order — the first one that fails stops it

1 · Written authorization exists	2 · Target named in the document	3 · Inside the time window	4 · Technique permitted	5 · Credentials and tools authorized
from the person entitled to bind the owner (Annex D for a real organization; the lab guide for lab work)	address or host name listed; excluded systems and everything unnamed are out	the authorization expires with the window; ask for a new one, never "finish quickly"	port scan, version detection, unauthenticated scan, passive baseline — exploitation and attack modes are out	credentialed scans need the credentials owner's written authorization; found credentials are reported, never used

All five pass → ALLOWED (the card lands inside the boundary) · gate 4 fails only because the guide says "ask" → ASK FIRST · any other failure → NOT ALLOWED

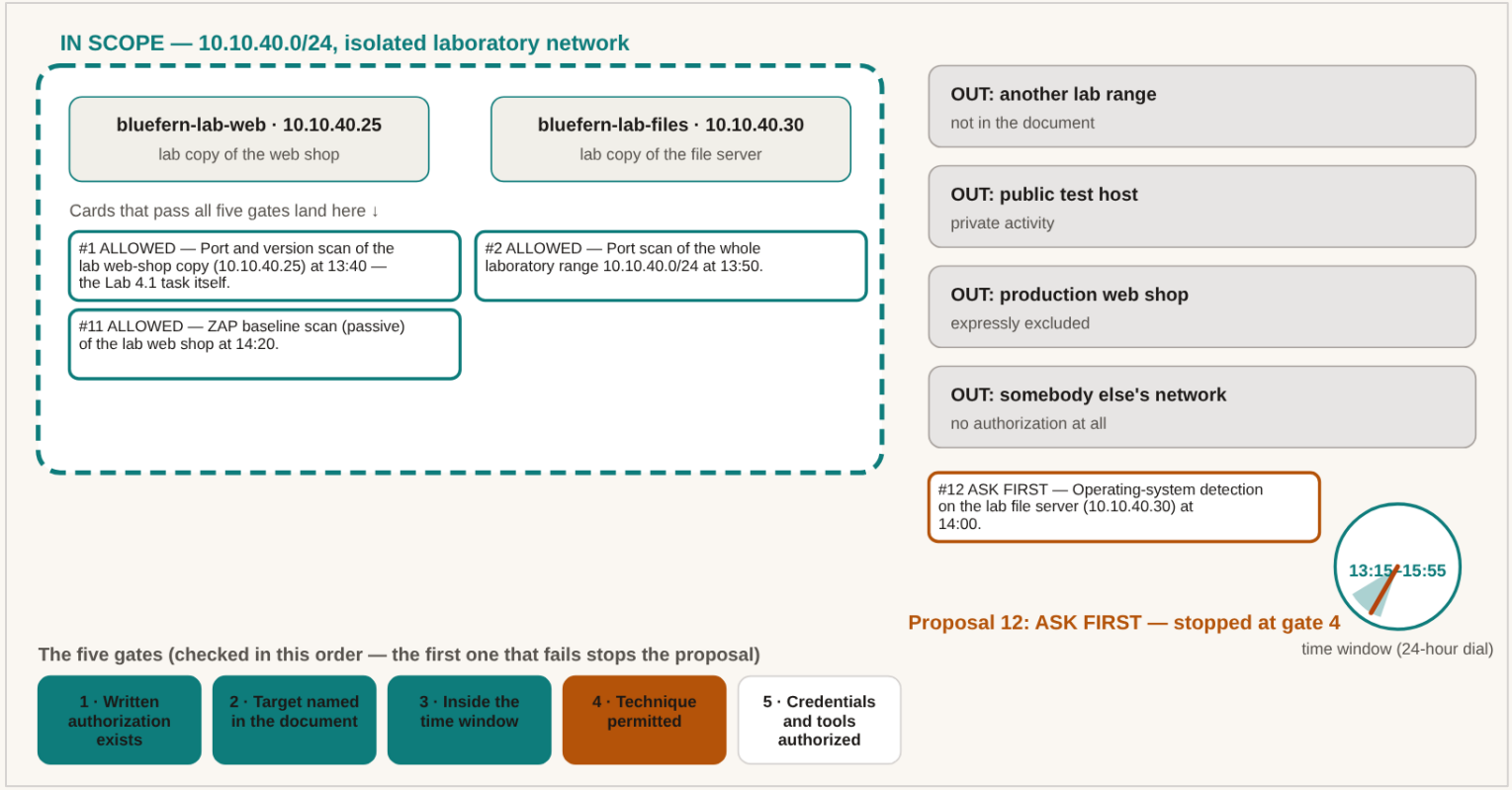
What applies when a gate fails — the law looks at authorization, not at intent

Canada — Criminal Code	s. 342.1 unauthorized use of a computer · s. 342.2 possession of a device (incl. software) to commit it · s. 430(1.1) mischief in relation to computer data · s. 184 interception
United States	Computer Fraud and Abuse Act, 18 U.S.C. § 1030
European Union	Directive 2013/40/EU on attacks against information systems
Spain	Ley Orgánica 10/1995 (Código Penal), arts. 197 and 264

Cyber.SoHo Legal Notice §7.6: a scan is an intrusive act, not an observation — it writes logs, triggers alerts, consumes resources, can crash fragile services, and is routinely reported as an attack. §7.7.5: outside the scope → stop, note the time, report.

CS 474 — Class 4 (Oct 15, 2026) · Cyber.SoHo Educational Hub · schematic 1 of 5

Interactive example 1 — Authorization Gate (twelve proposals, five gates)



- Allowed: the Lab 4.1 scan at 13:40 · the whole lab range · the passive baseline scan
- Not allowed: another lab range "because it answered" · the production address · somebody else's network · a public test host · exploitation · active attack mode · 16:30 · credentials found on the share
- Ask first: operating-system detection

Fun quiz — legal or not? (show of hands)

- Scanning your neighbour's Wi-Fi network from your apartment "just to see what is on it".
- Running the Lab 4.1 scan against the laboratory virtual machine at 13:40 today.
- Scanning bluefern.example, the scenario's production address, "because the company is fictional anyway".
- Answer with the five gates, not with intentions

Lesson 2 — Port & service scanning with Nmap

0:11–0:27 · **SYN** (synchronize flag — the first message of a TCP handshake) / connect / **UDP** (User Datagram Protocol) scans · open / closed / filtered · service & **OS** (Operating System) detection · reading a report

Ports, port scans and Nmap

- A **port** is a number from 0 to 65 535 that identifies, on one host, the program a packet is meant for — **TCP** (Transmission Control Protocol) and **UDP** (User Datagram Protocol) each have 65 536 ports
- A port is **open** when a service listens on it
- A **port scan** sends probe packets to a list of ports and interprets the answers: open, closed, or hidden behind a filter
- **Nmap** (Gordon Lyon, 1997; free, open source) is the reference tool; its reference guide defines every state
- By default Nmap probes the **1 000 most common TCP ports**, not all 65 536

nmap.org · Port Scanning Basics

Host discovery and the three-way handshake

- **Host discovery** first: **ICMP** (Internet Control Message Protocol) echo request · **TCP** (Transmission Control Protocol) **SYN** (synchronize flag — the first message of a TCP handshake) to 443 · **TCP ACK** (Acknowledgement) to 80 · **ICMP** timestamp request — any answer means "host is up"; the round-trip time becomes the latency in the report header
- The **three-way handshake**: client **SYN** → server **SYN**/ACK → client **ACK**; a port with nothing listening answers the **SYN** with **RST (Reset)**
- Every scan type is a different way of using (or not using) this handshake

SYN scan (-sS) — half-open (1/2)

- Send a **SYN (synchronize flag — the first message of a TCP handshake)** to the port and watch the answer
- **SYN/ACK** → **open** — Nmap immediately sends **RST** (Reset) so the connection is never completed (*half-open*)
- **RST** → **closed** — the host is reachable, nothing listens
- No answer after a retransmission, or an **ICMP** (Internet Control Message Protocol) "destination unreachable" error → **filtered** — a firewall dropped the probe
- Fast, works against any **TCP** (Transmission Control Protocol) stack; the application never logs it — but the **firewall and the IDS (Intrusion Detection System) see every probe**

Connect scan (-sT) and UDP scan (-sU) (2/2)

- **Connect scan**: Nmap asks the operating system for a normal connection — the handshake completes, then Nmap closes it; same three verdicts; used without administrator privileges; slower and **logged by the service**
- **UDP (User Datagram Protocol) scan**: no handshake exists — send a datagram and wait
- **ICMP** (Internet Control Message Protocol) port unreachable → **closed** · **UDP** answer → **open** · other **ICMP** unreachable → **filtered** · silence → **open|filtered**
- Hosts rate-limit **ICMP** errors (about 1 per second on Linux) → a full **UDP** scan takes hours; use short port lists (**DNS** (Domain Name System) 53, SNMP 161, NTP 123)

The six port states (Nmap Reference Guide)

State	Meaning	Produced by
open	A service is actively accepting TCP (Transmission Control Protocol) connections or UDP (User Datagram Protocol) datagrams on the port.	all
closed	The port is reachable (it answers probes) but nothing listens on it.	all
filtered	A packet filter prevents the probes from reaching the port; Nmap cannot tell whether it is open.	all
unfiltered	The port is reachable but Nmap cannot tell whether it is open or closed.	ACK (Acknowledgement) scan only (used to map firewall rules)
open|filtered	Nmap cannot tell whether the port is open or filtered (open ports give no answer to this probe type).	UDP, IP (Internet Protocol) protocol, FIN, NULL and Xmas scans
closed|filtered	Nmap cannot tell whether the port is closed or filtered.	IP ID (Identifier) idle scan only

Service / version detection (-sV) and OS detection (-O)

- Knowing port 21 is open tells you little; knowing it is **vsftpd 2.3.4** tells you everything — the next lesson compares versions with a vulnerability database
- **-sV**: for each open port Nmap completes a connection, reads the **banner**, sends protocol probes and matches the answers against thousands of signatures → SERVICE and VERSION columns
- **-O**: crafted packets; the **TCP** (Transmission Control Protocol)/IP stack's responses are matched against a fingerprint database; needs one open and one closed port
- In our laboratory, **OS** (Operating System) detection only when your lab option's guide says so

Reading a scan report — the course sample (1/2)

```
Nmap scan report for shop.bluefern.example (203.0.113.25)
Host is up (0.031s latency).
Not shown: 995 closed tcp ports (reset)
PORT     STATE SERVICE  VERSION
21/tcp   open  ftp      vsftpd 2.3.4
22/tcp   open  ssh      OpenSSH 7.2p2 Ubuntu 4ubuntu2.8
80/tcp   open  http     Apache httpd 2.4.18 ((Ubuntu))
443/tcp  open  ssl/http Apache httpd 2.4.18 ((Ubuntu))
3306/tcp open  mysql    MySQL 5.5.62
Service Info: OS: Linux
```

The course sample for the scenario's web shop (Appendix C.2): header · host state and latency · the "Not shown" summary of closed ports · one line per interesting port (PORT, STATE, SERVICE, VERSION) · the service-info footer. Read STATE first, then SERVICE, then VERSION; for each open port ask: should anyone on the Internet be able to talk to this?

Reading a scan report — what each line means (2/2)

- Host is up (0.031s latency) → discovery succeeded, 31 ms round trip · Not shown: 995 closed → only five interesting ports
- 21/tcp ftp vsftpd 2.3.4 → old version with a known backdoor (should the Internet talk to this? **no**)
- 22/tcp ssh OpenSSH 7.2p2 → remote administration open to the whole Internet, outdated (**no**)
- 80 and 443 Apache 2.4.18 → the web shop; plain **HTTP** (Hypertext Transfer Protocol) still served; outdated banner (**yes**, but fix)
- 3306/tcp mysql 5.5.62 → the customer database reachable from the Internet; support ended December 2018 (**no**)
- From the other side: one address asking about a thousand ports in seconds — what a firewall log or an **IDS** (Intrusion Detection System) recognises (Class 6)

Schematic 2 — How a SYN scan, a connect scan and a UDP scan decide a port's state

Schematic 2 — How a SYN scan, a connect scan and a UDP scan decide a port's state (Lesson 2)

Three scan types × three outcomes: the packets exchanged, the verdict Nmap prints, and what the target records. Definitions from the Nmap Reference Guide.

SYN scan (-sS) — default with administrator privileges

Open port



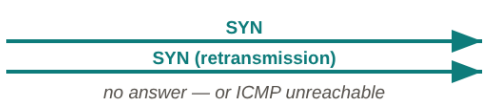
open
half-open: the service never logs a connection

Closed port



closed
reachable, nothing listens

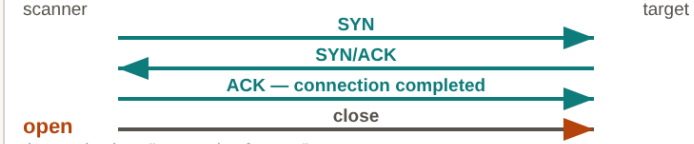
Filtered port



filtered
a packet filter dropped the probe

Connect scan (-sT) — default without privileges

Open port



open
the service logs "connection from …"

Closed port



closed
same verdict as the SYN scan

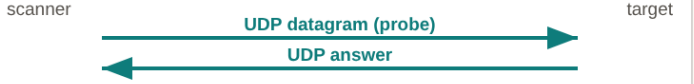
Filtered port



filtered
slower: the OS retries several times

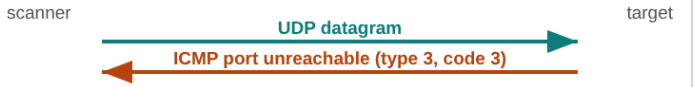
UDP scan (-sU) — no handshake

Open port



open
only when the service recognises the probe

Closed port



closed
hosts rate-limit these errors (≈1/s on Linux)

Open or filtered

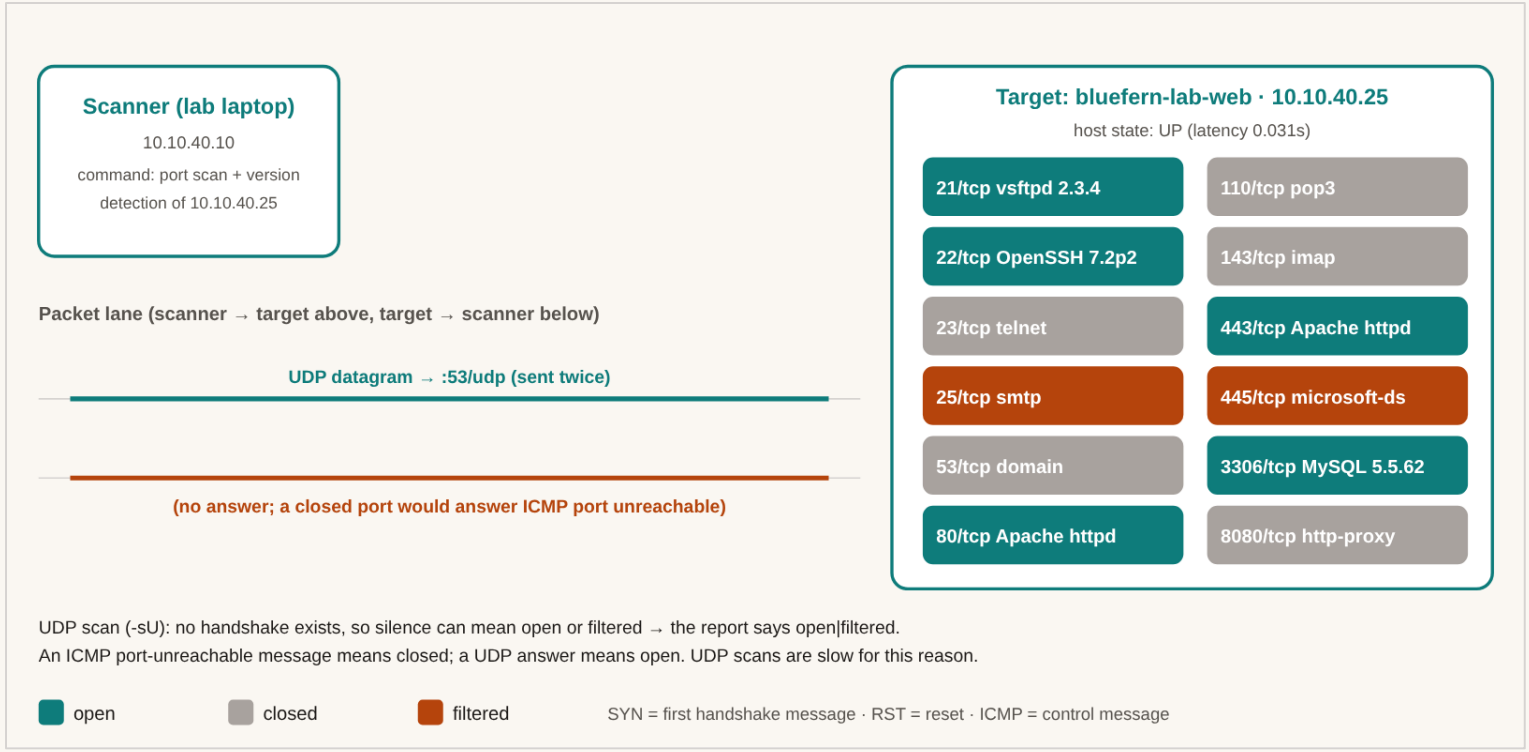


open|filtered
silence is ambiguous: no handshake exists

Six port states in the reference guide: open · closed · filtered · unfiltered (ACK scan only) · open|filtered · closed|filtered (idle scan only). After the port scan, version detection (-sV) completes a connection to each open port, reads the banner, sends protocol probes and matches the answers against a signature database → SERVICE and VERSION columns.

CS 474 — Class 4 (Oct 15, 2026) · Cyber.SoHo Educational Hub · schematic 2 of 5

Interactive example 2 — Port-Scan Simulator (no packet leaves the page)



- Two host-discovery exchanges · twelve **SYN** (synchronize flag — the first message of a TCP handshake) exchanges · five version-detection exchanges · the report assembled · connect-scan and **UDP** (User Datagram Protocol)-scan comparisons
- Downloads: the report (**TXT**, plain text file), the port table (**CSV**, Comma-Separated Values) — the Lab 4.1 scan summary

Lesson 3 — Vulnerability scanning & analysis: CVE, CVSS, false positives

0:27–0:43 · OpenVAS / Nessus-style scanners · **CVE** (Common Vulnerabilities and Exposures) identifiers · **CVSS** (Common Vulnerability Scoring System) scoring (computational) · verification

Vulnerability scanners — what they do and do not do

- Start from the open ports and versions; run **thousands of checks**: a probe per known weakness, or a version comparison
- Each match is a **finding**: identifier, severity score, description, evidence, recommended fix
- OpenVAS (Greenbone, open source) · Nessus (Tenable) · Qualys — same principle, different check databases
- **Unauthenticated scan**: from outside, sees what an attacker sees · **Credentialed scan**: logs in with an account you provide, reads installed packages — more accurate, fewer false alarms, written authorization of the credentials' owner required
- A scanner does **not** exploit: dangerous checks are off by default and stay off in our lab

OpenVAS · Nessus

CVE identifiers and the NVD

- **CVE** — Common Vulnerabilities and Exposures: the public catalogue of disclosed vulnerabilities, one identifier each: **CVE-year-number** (four or more digits)
- Operated by the **CVE** Program (MITRE); identifiers assigned by **CVE Numbering Authorities** — mostly the vendors themselves
- The shared vocabulary: vendor advisory, scanner finding, news article and your report all say CVE-2011-2523 and mean the same backdoor in vsftpd 2.3.4
- **NVD (National Vulnerability Database)** (**NIST**, National Institute of Standards and Technology) enriches each record: **CVSS** (Common Vulnerability Scoring System) score, affected products (**CPE**, Common Platform Enumeration), weakness type (**CWE**, Common Weakness Enumeration), references — where you verify what a finding is

cve.org · NVD · CVE-2011-2523

CVSS v3.1 — the eight base metrics (1/2)

- Maintained by **FIRST** (Forum of Incident Response and Security Teams); v3.1 (2019) is what scanners and the course use; v4.0 exists since November 2023
- **Exploitability group**: Attack Vector (Network / Adjacent / Local / Physical) · Attack Complexity (Low / High) · Privileges Required (None / Low / High) · User Interaction (None / Required)
- **Scope**: Unchanged / Changed — does the impact reach another component?
- **Impact group**: Confidentiality · Integrity · Availability — each None / Low / High (the triad of Class 1, measured one by one)
- The **base score** describes the vulnerability itself, independently of your environment

CVSS (Common Vulnerability Scoring System) v3.1 specification

The vector string, the score and the bands (2/2)

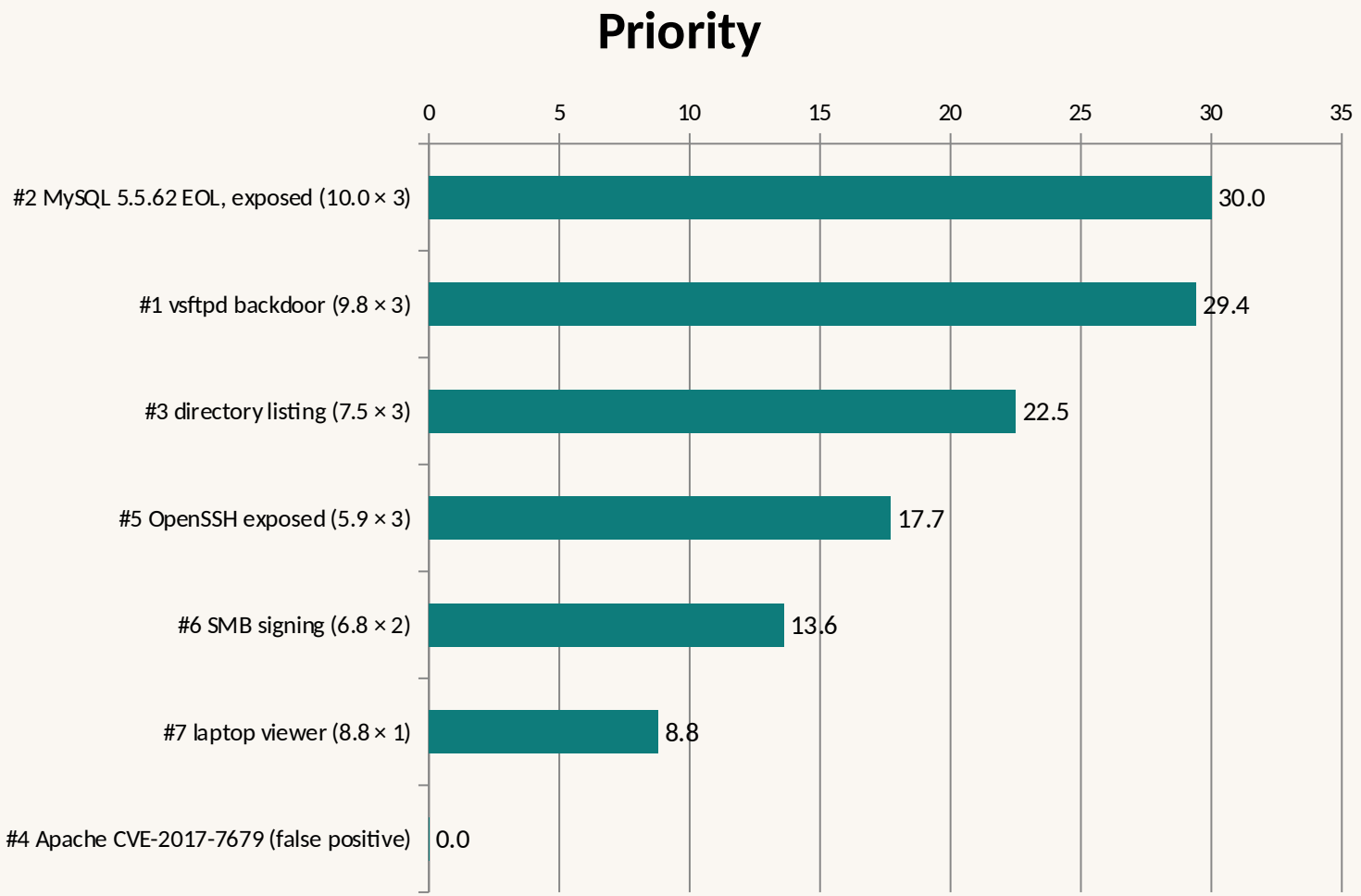
- **CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H** = network · low complexity · no privileges · no interaction · scope unchanged · high C, I and A → **9.8**: the worst shape
- Bands: None 0.0 · **Low** 0.1–3.9 · **Medium** 4.0–6.9 · **High** 7.0–8.9 · **Critical** 9.0–10.0
- AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N → needs a low-privilege account, reads data only → **6.5** Medium
- AV:A/AC:L/PR:N/UI:R/S:U/C:L/I:L/A:N → same network, a user must click, limited impact → **4.6** Medium

FIRST (Forum of Incident Response and Security Teams) calculator

How the number is computed — and the course's priority

- Weights: **AV** (Attack Vector) N 0.85 / A 0.62 / L 0.55 / P 0.2 · **AC** (Attack Complexity) L 0.77 / H 0.44 · **PR** (Privileges Required) N 0.85 / L 0.62 / H 0.27 (0.68 / 0.5 if Scope Changed) · **UI** (User Interaction) N 0.85 / R 0.62 · C, I, A: H 0.56 / L 0.22 / N 0
- **Exploitability** = 8.22 × **AV** × **AC** × **PR** × **UI**
- **ISS (Impact Sub-Score)** = 1 − (1 − C)(1 − I)(1 − A) · **Impact** = 6.42 × **ISS** (Scope Unchanged)
- **Base** = Roundup(min(Impact + Exploitability, 10)) — × 1.08 first when the Scope is Changed; 0 if Impact ≤ 0; always rounded **up** to one decimal
- Course convention: **priority = base score × asset criticality** (web shop 3 · file server 2 · laptop 1): 9.8 × 3 = 29.4 · 6.5 × 2 = 13.0 · 4.6 × 1 = 4.6

Seven findings of the lab scan — priority = CVSS × criticality



- A Medium on the web shop (#5, 17.7) outranks a High on a laptop (#7, 8.8) — that is what the multiplication is for
- The loudest finding on the scanner's screen (#4, 9.8) was not a finding at all

False positives and false negatives — a finding is a claim until verified

- **False positive**: reported, but does not exist · **False negative**: exists, but the scanner missed it
- Classic false positive: the banner says *Apache/2.4.18 (Ubuntu)* → the scanner reports CVE-2017-7679 (9.8), fixed upstream in 2.4.26
- Ubuntu **backports** fixes: the fix is in package 2.4.18-2ubuntu3.3 (USN-3340-1, June 2017) — same banner, not vulnerable
- Verification: the installed package version, or an authenticated scan that reads it
- False negatives: no check in the database · a filter hides the service · custom code nobody wrote a signature for → Lesson 4
- Read the **evidence** before the score: the evidence is what you verify, the score is what you compute with

Ubuntu — CVE-2017-7679 · NVD (National Vulnerability Database) — CVE-2017-7679

Schematic 3 — From a CVE record to a priority

Schematic 3 — From a CVE record to a priority: the chain of Lesson 3

Identifier → record → metrics → score and band → verification → × asset criticality → priority. Example: the vsftpd 2.3.4 backdoor on the lab web shop.

CVE identifier	NVD record (NIST)	CVSS v3.1 base metrics	Base score and band	Verify, then prioritize
CVE-2011-2523	**description · references**	**AV:N AC:L PR:N UI:N**	**9.8 = Roundup(min(5.873 + 3.887, 10))**	**evidence: banner + safe check → true**
one name per vulnerability	CVSS v3.1 base score + vector	S:U	band: Critical (9.0–10.0)	false positive? (backported fix) → 0
assigned by a CVE Numbering	affected products (CPE)	C:H I:H A:H	None 0.0 · Low 0.1–3.9	priority = base × criticality
Authority under the CVE Program	weakness type (CWE-78)	exploitability 3.887	Medium 4.0–6.9 · High 7.0–8.9	web shop 3 · file server 2 · laptop 1
(MITRE); format CVE-year-number	published date	impact 5.873 (ISS 0.915)	always rounded up to one decimal	9.8 × 3 = 29.4

The eight base metrics and their weights (FIRST, CVSS v3.1 specification)

Attack Vector (AV)	Network 0.85 · Adjacent 0.62 · Local 0.55 · Physical 0.2	from how far away can it be triggered?
Attack Complexity (AC)	Low 0.77 · High 0.44	does success depend on conditions outside the attacker's control?
Privileges Required (PR)	None 0.85 · Low 0.62 (0.68 if scope changed) · High 0.27 (0.5)	what account must the attacker already hold?
User Interaction (UI)	None 0.85 · Required 0.62	must a victim click or open something?
Scope (S)	Unchanged · Changed (impact formula changes, sum × 1.08)	does the impact reach another component?
Confidentiality / Integrity / Availability	High 0.56 · Low 0.22 · None 0	the triad of Class 1, measured one by one

Exploitability = 8.22 × AV × AC × PR × UI. ISS = 1 − (1−C)(1−I)(1−A). Impact = 6.42 × ISS (scope unchanged) or 7.52 × (ISS − 0.029) − 3.25 × (ISS − 0.02)^15 (changed). Base = Roundup(min(Impact + Exploitability, 10)) — ×1.08 first when the scope is changed; 0 if Impact ≤ 0.

Severity bands (v3.x: None = 0.0 exactly, then Low, Medium, High, Critical) — and the course's three worked examples



- AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H = 9.8 → ×3 (web shop) = 29.4
- AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N = 6.5 → ×2 (file server) = 13.0
- AV:A/AC:L/PR:N/UI:R/S:U/C:L/I:L/A:N = 4.6 → ×1 (laptop) = 4.6

A finding is a claim until verified: a version-based finding may be a false positive when the distribution backported the fix (Apache 2.4.18-2ubuntu3.3 fixes CVE-2017-7679 while the banner still says 2.4.18). Verify with the package version or an authenticated scan.

CS 474 — Class 4 (Oct 15, 2026) · Cyber.SoHo Educational Hub · schematic 3 of 5

Interactive example 3 — CVSS Scorer & Priority Ranker

Vector decoder — current finding

CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:U/C:H/I:H/A:H

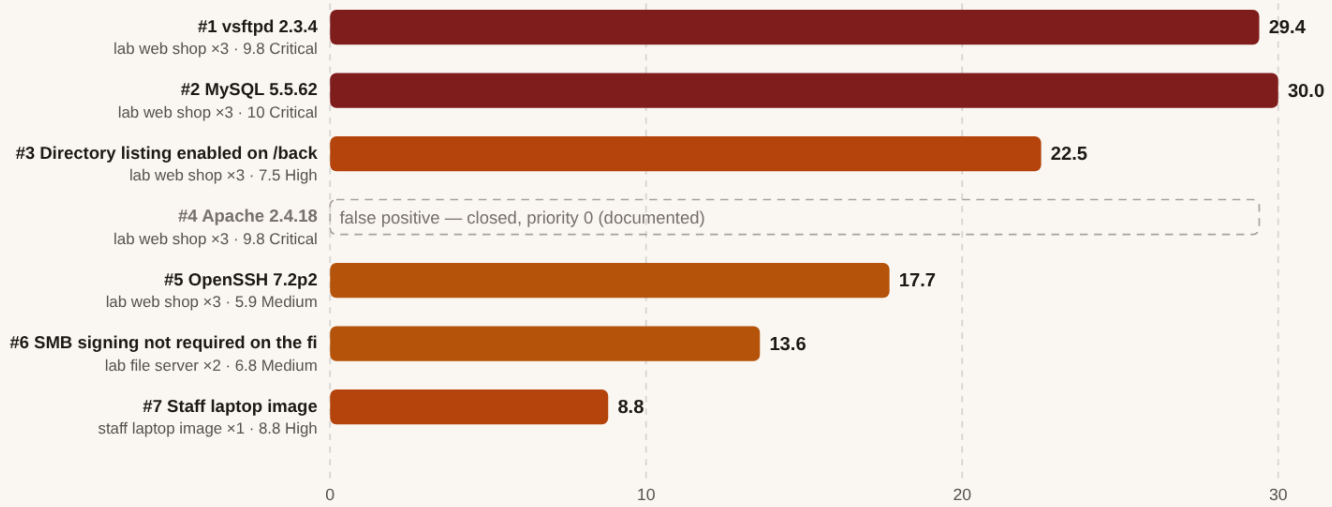
Attack Vector	Attack Complexity	Privileges Required	User Interaction	Scope	Confidentiality	Integrity	Availability
N · Network	L · Low	N · None	R · Required	U · Unchanged	H · High	H · High	H · High
0.85	0.77	0.85	0.62	unchanged	0.56	0.56	0.56

ISS = 1 − (1−0.56)(1−0.56)(1−0.56) = 0.9148 Impact = 6.42 × ISS = 5.8731

Exploitability = 8.22 × 0.85 × 0.77 × 0.85 × 0.62 = 2.8353 Base = Roundup(min((Impact + Exploitability), 10)) = 8.8

Ranking: 1. #2 MySQL 5.5.62 (30.0) · 2. #1 vsftpd 2.3.4 (29.4) · 3. #3 Directory listing enabled on /backup/ (22.5) · 4. #5 OpenSSH 7.2p2 (17.7) · 5. #…

Priority ladder — base score × asset criticality (web shop 3 · file server 2 · laptop 1)



- Four steps per finding: decode the vector · compute **ISS** (Impact Sub-Score), Impact, Exploitability and the rounded base score · verify · multiply by the criticality
- The ladder ranks the confirmed findings; the table has the Lab 4.2 columns (**CSV** (Comma-Separated Values) download)

Short computational — two vectors, the calculator, the priority

- File server (criticality 2): **CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N** → base score? band? priority?
- Staff laptop (criticality 1): **CVSS:3.1/AV:A/AC:L/PR:N/UI:R/S:U/C:L/I:L/A:N** → base score? band? priority?
- If time allows — web shop (criticality 3): CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H
- Use the **FIRST** (Forum of Incident Response and Security Teams) or **NVD** (National Vulnerability Database) calculator on your phone; multiply by the criticality; which is fixed first and why?

Lesson 4 — SAST, DAST & penetration-testing frameworks

0:43–0:51 · what each finds and what each misses · where each sits in the software life cycle

Static application security testing (SAST)

- Analyses the **source code** (or compiled code) **without running it**
- Pattern rules ("a string literal assigned to a variable named *pass*") and data-flow rules ("does request input reach a query, an output or a command without validation?")
- Runs where the code lives: the editor and **every commit** in the build pipeline — a flaw found there costs minutes
- **Finds**: hard-coded credentials, injection sinks, dangerous functions, weak cryptography, missing validation
- **Misses**: server configuration, **TLS** (Transport Layer Security) deployment, logic that needs reasoning · many **false positives**
- Semgrep · Bandit (Python) · Psalm / PHPStan (**PHP**, PHP: Hypertext Preprocessor) · SonarQube

Semgrep · Bandit

Dynamic application security testing (DAST)

- Tests the **running application from outside**, as a black box, without the code: crawls pages, submits inputs, inspects responses, headers, cookies, error messages
- Reference tool: **OWASP (Open Worldwide Application Security Project) ZAP (Zed Attack Proxy)** (free) — the **baseline scan** crawls for a minute and runs only passive rules (no attack): the one mode our scope document permits; the **active scan** sends attack payloads and needs a separate authorization
- **Finds**: reflected input, missing security headers, cookie flags, verbose errors, **HTTP** (Hypertext Transfer Protocol) without **HTTPS** (Hypertext Transfer Protocol Secure), outdated banners
- **Misses**: paths the crawler never reaches, secrets that live only in the code, logic flaws · needs a deployed test copy

OWASP ZAP · ZAP baseline scan

Penetration-testing frameworks — methodology and toolkit

- The word *framework* means two things
- **Methodology**: **NIST** (National Institute of Standards and Technology) **SP** (Special Publication) 800-115 (planning, discovery, attack, reporting) · **PTES** (Penetration Testing Execution Standard) (seven phases from pre-engagement — the rules of engagement — to reporting)
- **Toolkit**: Metasploit — a library of modules that exploit known weaknesses; for that very reason **out of scope in Class 4**
- A penetration test is **human-led and time-boxed**: chains what the tools reported, proves impact, finds logic and design errors no rule can express
- **Misses**: breadth — a two-day test does not enumerate every instance; results depend on the tester

NIST SP 800-115 · PTES · Metasploit

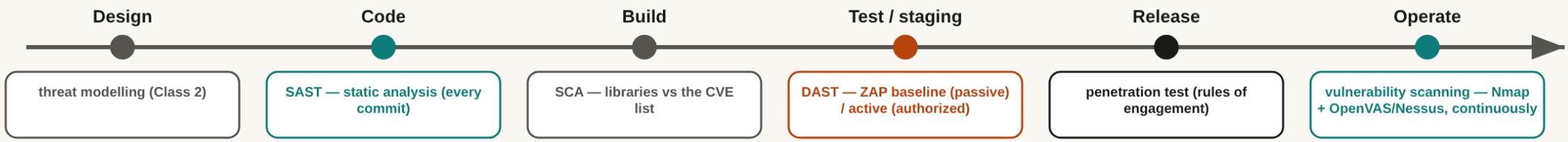
Capabilities and limits — the Lab 4.3 table

Lens	Finds	Misses	When to use
Static analysis (SAST, Static Application Security Testing)	Hard-coded secrets; request input reaching queries, output or commands; dangerous functions; weak cryptography; missing validation	Server configuration and TLS (Transport Layer Security) deployment; logic that needs reasoning; anything outside the repository	While coding and at every commit (build pipeline)
Dynamic analysis (DAST, Dynamic Application Security Testing)	Reflected input without encoding; missing security headers; cookie flags; verbose errors; HTTP (Hypertext Transfer Protocol) without HTTPS (Hypertext Transfer Protocol Secure); injection where a parameter is reachable	Paths the crawler never reaches; secrets that live only in the code; logic and design flaws	Against the test / staging deployment before release; periodically in operation
Penetration test (human)	Logic and design errors; real impact (administrator access); password reuse across hosts; confirmation of tool findings	Breadth: a time-boxed test does not enumerate every instance; results depend on the tester	Before go-live and after major changes
Vulnerability scanning (Lessons 2–3)	Known vulnerabilities in known products (CVE, Common Vulnerabilities and Exposures), exposed services, end-of-life software, configuration findings	Custom code; what a filter hides; weaknesses without a signature	Continuously in operation

Schematic 4 — SAST, DAST and penetration testing on the software life cycle

Schematic 4 — SAST, DAST and penetration testing on the software life cycle (Lesson 4)

Where each tool family sits, what it looks at, what it finds, what it misses, and the authorization it needs.



Static analysis (SAST)

looks at: the source code, without running it

finds
- hard-coded secrets
- input reaching queries / output / commands
- dangerous functions, weak cryptography
- missing validation

misses
- server configuration, TLS deployment
- logic that needs reasoning
- anything outside the repository

authorization / limits

needs the code; many false positives — verify each

examples

Semgrep · Bandit · Psalm / PHPStan · SonarQube

Dynamic analysis (DAST)

looks at: the running application, from outside (black box)

finds
- reflected input without encoding
- missing security headers, cookie flags
- verbose errors, HTTP without HTTPS
- injection where a parameter is reachable

misses
- paths the crawler never reaches
- secrets that live only in the code
- logic and design flaws

authorization / limits

needs a deployed test copy; active mode = attack traffic = separate authorization

examples

OWASP ZAP (baseline / active) · Burp Suite

Penetration test (human)

looks at: meaning — chains findings, proves impact

finds
- logic and design errors (predictable tokens)
- real impact of a flaw (administrator access)
- reuse of a password across hosts
- what a tool reported, confirmed

misses
- breadth: not every instance of a pattern
- results depend on the tester and the time box

authorization / limits

needs written rules of engagement (NIST SP 800-115, PTES); toolkits such as Metasploit are out of scope in Class 4

examples

methodology: NIST SP 800-115 · PTES · OWASP Testing Guide

CS 474 — Class 4 (Oct 15, 2026) · Cyber.SoHo Educational Hub · schematic 4 of 5

Interactive example 4 — Three Lenses on One Login Page

bluefern_login_demo.php — 30 lines, seven planted weaknesses (A–G)

```
1  <?php
2  // BlueFern Trading Inc. - demo login page (laboratory copy, deliberately flawed)
3  $db_host = 'localhost';
4  $db_user = 'shop';
5  $db_pass = 'BlueFern2019!'; // A: hard-coded credential
6  $conn = mysqli_connect($db_host, $db_user, $db_pass, 'shop');
7  if (!$conn) { die('DB error: ' . mysqli_connect_error()); } // E: error shown to the vis…
8  $user = $_POST['user'];
9  $pass = $_POST['pass'];
10 $sql = "SELECT id, role FROM users WHERE user='" . $user . "' AND pass='" . $pass . "'"; …
11 $res = mysqli_query($conn, $sql);
12 if (!$res) { die('Query failed: ' . mysqli_error($conn)); } // E: error shown to the vi…
13 if (mysqli_num_rows($res) === 1) {
14 $row = mysqli_fetch_assoc($res);
15 $sid = md5($user . time()); // F: predictable session identifier
16 setcookie('session', $sid); // D: cookie without Secure / HttpOnly
17 echo '<h1>Welcome back, ' . $user . '</h1>'; // C: input echoed without encoding
18 if ($row['role'] === 'admin') {
19 echo '<a href="/admin/">Open the administration panel</a>';
20 }
21 } else {
22 echo '<p>Login failed for ' . $user . '</p>'; // C: input echoed without encoding
23 }
24 if ($_SERVER['REQUEST_SCHEME'] === 'http') {
25 // TODO: redirect to https - not implemented // G: served over plain HTTP
26 }
27 // password reset link
28 $reset = '/reset.php?token=' . md5(time()); // F: predictable reset token
29 mysqli_close($conn);
30 ?>
```

Lens × weakness matrix

		SAST	DAST	PENTEST
A	Hard-coded databas password	found	missed	found
B	Query built by concatenation; plain	found	found	found
C	User input echoed without output encoc	found	found	partial
D	Session cookie without Secure and	found	found	partial
E	Database error messages shown to	found	found	partial
F	Predictable session identifier and reset t	partial	missed	found
G	Served over plain HTTP, no redirect to	missed	found	found

found = teal · partial = amber · missed = grey

Summary: SAST reads text, DAST reads behaviour, the tester reads meaning. Use all three at different moments of the life cycle.

- Seven planted weaknesses (A–G) on a 30-line login page
- **SAST** (Static Application Security Testing) found A B C D E, partial F, missed G · **DAST** (Dynamic Application Security Testing) found B C D E G, missed A F · the tester found A B F G, noted C D E in passing

Flash-card blitz — six terms, one sentence each

- SAST
- DAST
- Penetration test
- CVE
- CVSS
- False positive

Lesson 5 — From findings to fixes

0:51–1:00 · prioritizing (**CVSS** (Common Vulnerability Scoring System) × asset criticality) · patching & mitigation · the report a manager will read

Prioritize — and two refinements

- Course rule: **priority = CVSS (Common Vulnerability Scoring System) base score × asset criticality** (web shop 3 · file server 2 · laptop 1)
- **Exposure**: a finding on a service the whole Internet can reach (ports 21, 22, 3306 of the web shop) outranks the same score on an internal host
- **Exploit availability**: **CISA** (Cybersecurity and Infrastructure Security Agency)'s Known Exploited Vulnerabilities catalogue (confirmed exploitation in the wild) · **FIRST** (Forum of Incident Response and Security Teams)'s **EPSS** (Exploit Prediction Scoring System) (probability of exploitation in the next 30 days) — a **CVE** (Common Vulnerabilities and Exposures) on either list moves up

CISA KEV (Known Exploited Vulnerabilities) · EPSS

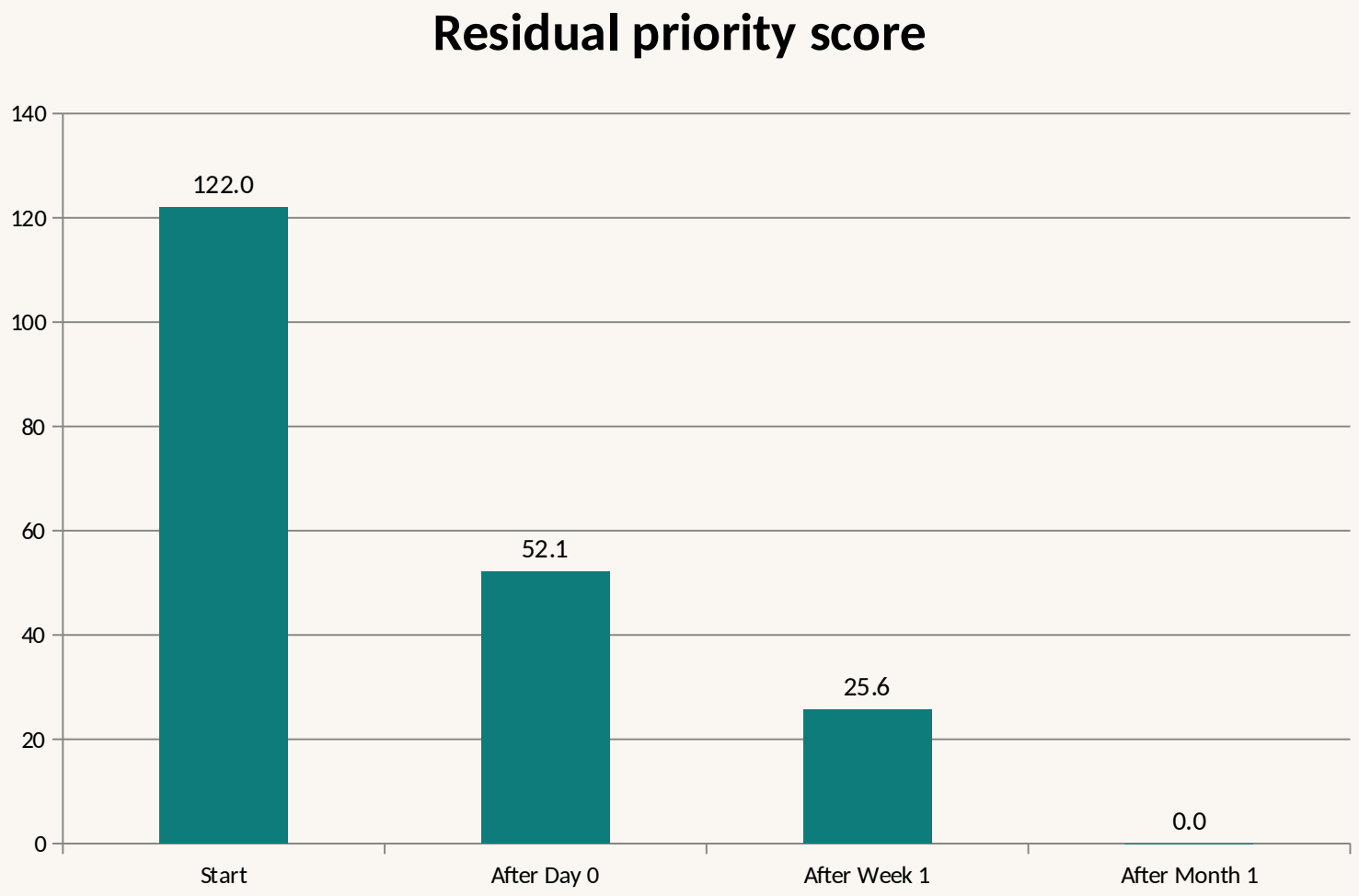
Decide — there are only five decisions

- **Remove** — the service is not needed (**FTP** (File Transfer Protocol) "for quick uploads"): switch it off — cheapest and permanent
- **Mitigate** — the real fix cannot happen today: a firewall rule, a configuration change, segmentation; then schedule the fix
- **Patch** — the vendor's update, after a backup, in a maintenance window; routine cycle plus emergency patches; check for backports first
- **Migrate** — end of life means no patch will ever come (MySQL 5.5 ended 31 December 2018): the only fix is a supported release
- **Accept** — no fix and no cheap mitigation: in writing, with reason, owner and review date
- Every decision gets an **owner**, an **effort** and a **deadline**: today · this week · this month · next quarter

Verify, then write the report a manager will read

- **Verify**: a fix is done when a **rescan** shows the finding gone; keep the before-and-after evidence
- Binder Part 4, in order: **scope and authorization statement** · **executive summary** (4 sentences: what was assessed, how bad, first action, cost and verification date — in business terms) · **top-5 table** (finding, asset, **CVE** (Common Vulnerabilities and Exposures), **CVSS** (Common Vulnerability Scoring System), priority, fix, owner, deadline) · **tool-comparison table** · **plan** with residual risk · **verification** · appendices with raw output
- "The customer database can be reached from the Internet" — not "3306/tcp open mysql 5.5.62"
- One page first; everything else attached — the manager reads page one, the engineer reads the appendix

The plan — residual priority after each deadline (Lesson 5 example)



- Day 0 (4 hours): firewall the database and bind it to the private network · switch **FTP** (File Transfer Protocol) off · remove the backup folder → 57 % of the score removed
- Week 1: update and restrict **SSH** (Secure Shell) · patch the laptop image · Month 1: migrate the database; require **SMB** (Server Message Block) signing after testing

Schematic 5 — From findings to fixes: the decision rules and the one-page report

Schematic 5 — From findings to fixes: the decision rules and the one-page report (Lesson 5)

Prioritize → decide → verify → report. Priority = CVSS base score × asset criticality, refined by exposure and exploit availability (CISA KEV, EPSS).

Decision rules, asked in order

Is the service needed at all?	**no → REMOVE (Day 0)**	e.g. FTP "for quick uploads" → switch off, use SFTP
Exposed to the network and Critical, or in the KEV catalogue?	**yes → MITIGATE today, then fix within the month**	firewall rule, bind to the private network, segmentation
Vendor fix available, no side effects?	**yes → PATCH (Week 1)**	backup first; check whether the fix is already backported
Software at end of life?	**yes → MIGRATE (Month 1)**	no patch will ever come: MySQL 5.5 ended 31 Dec 2018
Fix changes behaviour that must be tested?	**yes → HARDEN after testing (Month 1)**	SMB signing: the warehouse scanners may stop connecting
Otherwise	**→ ACCEPT in writing, review next quarter**	reason, owner, review date — documented

Every decision carries an owner, an effort estimate and a deadline (Day 0 · Week 1 · Month 1 · Quarter). Verify: a fix is done when a rescan shows the finding gone; keep the before-and-after evidence.

Vulnerability Assessment Report — page 1

1 · Scope and authorization statement
systems, window, techniques, contact — Lesson 1

2 · Executive summary (4 sentences)
what was assessed · how bad · first action · cost and verification date — in business terms

3 · Top-5 findings table
finding · asset (crit.) · CVE · CVSS · priority · fix · owner · deadline

4 · Tool comparison
SAST / DAST / penetration test — finds · misses · when to use (Lab 4.3)

5 · Plan and residual risk score
122.0 → 52.1 after Day 0 → 25.6 after Week 1 → 0 after Month 1

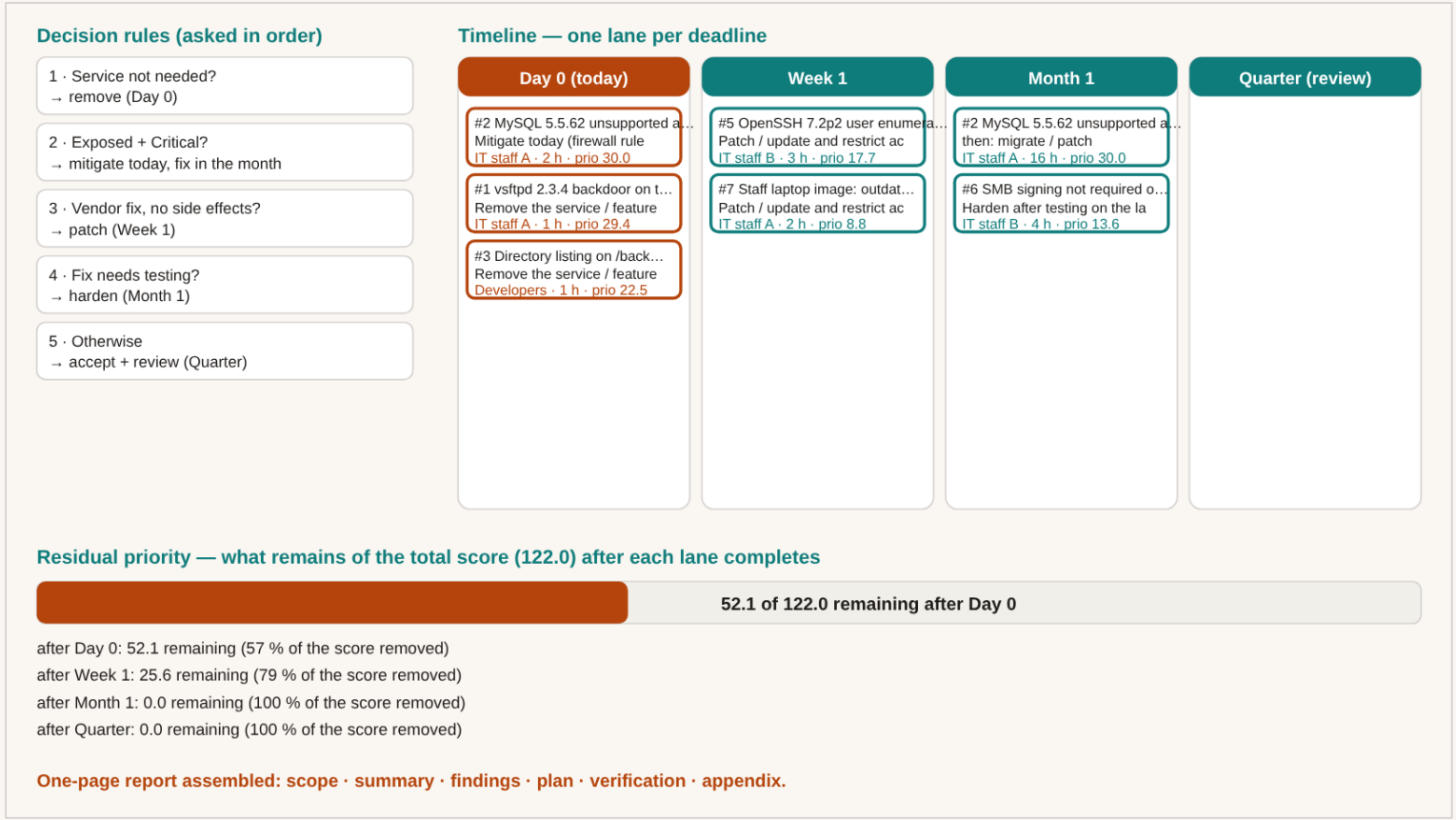
6 · Verification
rescan results, package versions, closed ports

Appendices
raw scan output · scanner excerpts · evidence · false positives documented

The manager reads page 1; the engineer reads the appendix; both must find what they need.

CS 474 — Class 4 (Oct 15, 2026) · Cyber.SoHo Educational Hub · schematic 5 of 5

Interactive example 5 — Fix Planner & One-Page Report Builder



- Ten steps: load the ranked findings · decide one at a time (the rule that fires lights up; the card lands in its lane) · residual priority · executive summary · the report in the Binder Part 4 layout (Markdown download)

Summary of the hour — five sentences

- A scan is lawful only inside a **written scope**, a **time window** and a list of **permitted techniques** — the law looks at authorization, not intent
- A port scan sends a **SYN (synchronize flag — the first message of a TCP handshake)** per port and reads **open, closed or filtered** from the answer; **version detection** turns the port into a product
- A **CVE (Common Vulnerabilities and Exposures)** is the vulnerability's name, **CVSS (Common Vulnerability Scoring System)** its score, **priority** = score × asset criticality — and every finding is **verified** before it is reported
- **SAST (Static Application Security Testing)** reads code, **DAST (Dynamic Application Security Testing)** reads behaviour, a **penetration tester** reads meaning — each at its moment of the life cycle
- Findings become fixes through **decisions** with owners and deadlines, **verified by a rescan**, written on **one page** a manager will read

Fun quiz — first action? (show of hands) · then the Zen break

- The scanner reports that the customer database (MySQL 5.5.62, end of life) is reachable from the Internet.
- The scanner reports CVE-2017-7679 (9.8) on "Apache/2.4.18 (Ubuntu)".
- Two findings remain for the week: a Medium (5.9) on the web shop and a High (8.8) on one laptop.
- Zen break (2 min): stand up, three slow breaths, eyes on a point six metres away

Binder Part 4 and next week

Binder Part 4 — Vulnerability Assessment Report

Scope & authorization statement · scan summary · top-5 table (**CVE** (Common Vulnerabilities and Exposures) / **CVSS** (Common Vulnerability Scoring System) / priority / fix) · tool-comparison table — due Wednesday, October 21, 23:59.

Hours 2–3 today

Lab 4.1 scan · Lab 4.2 analyse the vulnerability-scan excerpt (top-5, **CVE**, **CVSS**, priority, fix) · Lab 4.3 compare **SAST** (Static Application Security Testing), **DAST** (Dynamic Application Security Testing) and a penetration test (sheets provided separately).

At home

Easttom, Chapter 11. Keep the scan output you started from — the before-and-after pair is what the Mid-Term asks for.

Next Thursday — Class 5

Inside the Machine: Operating System Security & Malware (Easttom Ch. 5 + 9) — Binder Part 5, Malware Playbook & Hardening.

Links used in this lecture (1/2)

Close

- Cyber.SoHo — Legal Notice, Terms of Use and Acceptable Use Policy (v2.0)
- Nmap — the Network Mapper (official site)
- Nmap Reference Guide — Port Scanning Basics (the six port states)
- Nmap Reference Guide — Port Scanning Techniques (SYN (synchronize flag — the first message of a TCP handshake), connect, UDP (User Datagram Protocol) and the others)
- Nmap — Legal Issues
- Greenbone OpenVAS — Open Vulnerability Assessment Scanner
- Tenable Nessus
- CVE (Common Vulnerabilities and Exposures) Program (cve.org)
- NIST (National Institute of Standards and Technology) National Vulnerability Database (NVD)
- NVD (National Vulnerability Database) — CVE-2011-2523 (vsftpd 2.3.4 backdoor)
- NVD — CVE-2017-7679 (Apache httpd mod_mime buffer over-read)
- Ubuntu Security — CVE-2017-7679 package status
- NVD — CVE-2016-6210 (OpenSSH user enumeration)
- FIRST (Forum of Incident Response and Security Teams) — CVSS (Common Vulnerability Scoring System) v3.1 Specification Document
- FIRST — CVSS v3.1 Calculator
- CISA (Cybersecurity and Infrastructure Security Agency) — Known Exploited Vulnerabilities (KEV) Catalog
- FIRST — Exploit Prediction Scoring System (EPSS)

Links used in this lecture (2/2)

Close

- MITRE CWE — Common Weakness Enumeration
- OWASP (Open Worldwide Application Security Project) ZAP — Zed Attack Proxy
- OWASP ZAP — Baseline Scan documentation
- OWASP Top 10
- Semgrep (static analysis)
- Bandit (static analysis for Python)
- Metasploit Framework (official site)
- NIST (National Institute of Standards and Technology) SP (Special Publication) 800-115 — Technical Guide to Information Security Testing and Assessment
- PTES — Penetration Testing Execution Standard
- endoflife.date — MySQL support dates
- Criminal Code of Canada, s. 342.1 — Unauthorized use of computer
- US (United States) Computer Fraud and Abuse Act, 18 U.S.C. § 1030
- EU (European Union) Directive 2013/40/EU on attacks against information systems
- Spain — Ley Orgánica 10/1995 (Código Penal)
- Canadian Centre for Cyber Security (CCCS)
- Pearson — Easttom, Computer Security Fundamentals (4th ed.)
- VirusTotal — URL (Uniform Resource Locator) scanner
- Google Safe Browsing — site status

Legal notice, terms of use and acceptable use

Close

Educational material — Cyber.SoHo / Asen Grozdanov. © 2026 Cyber.SoHo. All rights reserved. For use in an isolated, authorized lab environment only, against the targets this exercise designates. Applying these tools or techniques to any other system, network or data without the owner's express written authorization may be a criminal offence (Criminal Code, ss. 342.1, 342.2, 430(1.1) and 184) and is strictly prohibited. You alone are responsible for your actions and your equipment. Full terms: Cyber.SoHo Legal Notice, Terms of Use and Acceptable Use Policy, v2.0, section 7.

All scanning, testing and "attack" exercises in this course are performed exclusively on the lab systems and the fictional BlueFern Trading Inc. scenario provided, with explicit authorization. Unauthorized access to, or interference with, computer systems is a criminal offence — for example under the Canadian Criminal Code s. 342.1, the **US** (United States) Computer Fraud and Abuse Act 18 U.S.C. § 1030, **EU** (European Union) Directive 2013/40/EU and Spain's Ley Orgánica 10/1995 — regardless of intent. Never test a system you do not own or do not have written permission to test. No **OSINT** (Open-Source Intelligence) on real people or real organizations is ever part of this course.

Full terms: Cyber.SoHo Legal Notice, Terms of Use and Acceptable Use Policy (v2.0) — sections 3, 4, 6, 7.6, 7.7, 8 and 9; Annex C (link safety); Annex D (written authorization template). The student acknowledgement (Annex A) is provided as a separate file. This is a computer-security course: every technique is taught so that systems, the software on them and the data they hold can be assessed and defended.

https://cyber-soho.github.io/prevmon/00.01.%20cyber.soho%20-%20legal.html